CAP 5636 – Midterm Exam

Date: ______________

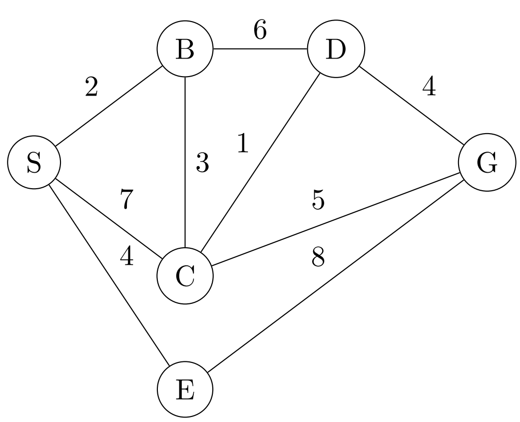
Name: ____

Instructions:

- This exam is open book and open notes. Textbooks and notes on tablet devices are acceptable but they must be put into airplane mode. No device with a keyboard is acceptable.
- Allotted time is 75 minutes.
- It is strongly recommended that you use a pencil, such that you can make corrections if needed.
- You can continue a solution on the back of the page. Do not add new pages.
- Show all work for partial credit.
- Note that the points add up to 100 + 20 bonus points.

Problem 1: Search Algorithms (35 pts)

Consider the following state space graph where you want to find a path from Start (S) to Goal (G). Each edge shows its cost.



Part A (15 pts): Trace through the execution of **Breadth-First Search (BFS)** starting from S. Show the fringe at each step and indicate which path to the goal is found. Remember that BFS uses a FIFO queue.

Format: Step N: Expanded: [node], Fringe: {list of nodes}, Path found (if goal reached): [path]

Part B (10 pts): Now trace through **Uniform Cost Search (UCS)** on the same graph. Show the fringe with costs at each step.

Format: Step N: Expanded: [node:cost], Fringe: {node1:cost1, node2:cost2, ...}

Part C (10 pts): If we were to use **Depth-First Search (DFS)** on this graph (assume alphabetical ordering for tie-breaking), would it find the optimal path? Explain why or why not in 2-3 sentences. What are the space complexity advantages of DFS compared to BFS?

Problem 2: A* Search and Heuristics (35 pts)

Consider the search problem shown in the graph below, where we want to find the optimal path from S to G. The number on each edge represents the cost, and h-values (heuristic estimates to the goal) are shown for each node.

Node	S	A	B	C	D	G
h(n)	7	5	6	2	3	0

Edges:

- S $\rightarrow$ A: 2
- S $\rightarrow$ B: 3
- A $\rightarrow$ C: 4
- A $\rightarrow$ D: 1
- B $\rightarrow$ C: 2
- B $\rightarrow$ D: 5
- C $\rightarrow$ G: 3
- D $\rightarrow$ G: 4

Part A (15 pts): Execute A* tree search on this graph. At each step, show:

- The node being expanded
- The f-value calculation (f = g + h)
- The fringe with f-values
- When you reach the goal, indicate the optimal path found

Part B (10 pts): Is the given heuristic **admissible**? Is it **consistent**? Prove or disprove each property. Show your work.

Part C (10 pts): Suppose we modify the heuristic by setting h(D) = 6 instead of 3. Would A* still find the optimal solution? Would it be admissible? Would it be consistent? Explain your reasoning.

Problem 3: Probability and Bayes Nets (30 pts)

A smart home system monitors three variables:

- **M**: Motion detected (binary: yes/no)
- **L**: Lights are on (binary: on/off)
- **A**: Alarm triggered (binary: yes/no)

The system works as follows: Motion can cause lights to turn on. Both motion and lights being on can trigger the alarm.

Given probabilities:

- $P(+m) = 0.3$
- $P(+l| + m) = 0.9$
- $P(+l| - m) = 0.2$
- $P(+a| + m, +l) = 0.8$
- $P(+a| + m, -l) = 0.6$
- $P(+a| - m, +l) = 0.1$
- $P(+a| - m, -l) = 0.01$

Part A (10 pts): Draw the Bayesian Network for this scenario. Include the conditional probability tables.

Part B (10 pts): Calculate $P(+a)$. Show all your work using marginalization.

Part C (10 pts): Using Bayes' rule, calculate $P(+m| + a)$. Show all steps in your calculation.

Problem 4: Graph Search and Optimality (20 pts BONUS)

Consider implementing A* graph search (as opposed to tree search) with an admissible but **inconsistent** heuristic.

Part A (10 pts): Give a specific example of a small graph (4-5 nodes) with costs and an admissible but inconsistent heuristic. Show that your heuristic is:

1. Admissible (never overestimates)
2. Inconsistent (violates the consistency condition)

Part B (10 pts): Using your example from Part A, demonstrate how A* graph search might fail to find the optimal solution. Trace through the algorithm execution and explain exactly where and why the suboptimality occurs.

Answer Key

Problem 1: Search Algorithms

Part A - BFS Trace:

1. Step 1: Expanded: S, Fringe: {B, C, E}
2. Step 2: Expanded: B, Fringe: {C, E, D}
3. Step 3: Expanded: C, Fringe: {E, D, G}
4. Step 4: Expanded: E, Fringe: {D, G}
5. Step 5: Expanded: D, Fringe: {G}
6. Step 6: Expanded: G, Path found: S → C → G

BFS finds the path with the fewest edges (S → C → G), which has 2 edges.

Part B - UCS Trace:

1. Step 1: Expanded: S:0, Fringe: {B:2, E:4, C:7}
2. Step 2: Expanded: B:2, Fringe: {E:4, C:5, C:7, D:8}
3. Step 3: Expanded: E:4, Fringe: {C:5, D:6, C:7, D:8, G:12}
4. Step 4: Expanded: C:5, Fringe: {D:6, C:7, D:8, G:10, G:12}
5. Step 5: Expanded: D:6, Fringe: {C:7, D:8, G:10, G:12}
6. Step 6: Expanded: C:7 (skipped - already visited)
7. Step 7: Expanded: D:8 (skipped - already visited)
8. Step 8: Expanded: G:10, Optimal path found: S → B → C → G with cost 10

Part C - DFS Analysis:

DFS with alphabetical ordering would explore: S → B → C → D → G, finding a path with cost 15. This is NOT optimal (optimal is 10). DFS doesn't guarantee optimality because it commits to the first complete path it finds without considering costs.

Space complexity advantage: DFS only needs $O(bm)$ space where b is branching factor and m is maximum depth, while BFS needs $O(b^d)$ where d is the depth of the shallowest goal. DFS only stores the current path and siblings along that path, while BFS stores entire levels of the search tree.

Problem 2: A* Search and Heuristics

Part A - A* Trace:

1. Initial: Fringe: {S: f=0+7=7}
2. Expand S:
    - Add A: g=2, h=5, f=7

- Add B: g=3, h=6, f=9

  Fringe: {A:7, B:9}

3. Expand A (f=7):
   - Add C: g=2+4=6, h=2, f=8
   - Add D: g=2+1=3, h=3, f=6

   Fringe: {D:6, C:8, B:9}

4. Expand D (f=6):
   - Add G: g=3+4=7, h=0, f=7

   Fringe: {G:7, C:8, B:9}

5. Expand G (f=7): Goal reached! Optimal path: S → A → D → G with cost 7

Part B - Admissibility and Consistency:
Admissibility: A heuristic is admissible if $h(n) \leq h^*(n)$ for all nodes, where $h^*(n)$ is the true cost to goal.

- h(S) = 7, true cost = 7 (S → A → D → G) ✓
- h(A) = 5, true cost = 5 (A → D → G) ✓
- h(B) = 6, true cost = 5 (B → C → G) **NOT admissible!**
- h(C) = 2, true cost = 3 (C → G) ✓
- h(D) = 3, true cost = 4 (D → G) ✓
- h(G) = 0, true cost = 0 ✓

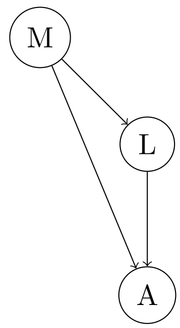
The heuristic is **NOT admissible** because h(B) = 6 ¿ 5.
Consistency: A heuristic is consistent if $h(n) \leq c(n, n') + h(n')$ for all edges.
Check edge B → C: h(B) = 6, c(B,C) = 2, h(C) = 2 $6 \leq 2 + 2 = 4$ is FALSE.
The heuristic is **NOT consistent**.
Part C - Modified Heuristic:
With h(D) = 6 instead of 3:

- **Admissibility:** h(D) = 6 but true cost = 4, so 6 ¿ 4. NOT admissible.
- **Consistency:** Check A → D: h(A) = 5, c(A,D) = 1, h(D) = 6. We need $5 \leq 1 + 6 = 7$ ✓. But other edges might fail.
- **Optimality:** Since the heuristic is not admissible, A* is NOT guaranteed to find the optimal solution.

In the trace, when we expand A, D would have f = 3 + 6 = 9, so we'd explore C first (f=8), potentially finding a suboptimal path.

Problem 3: Probability and Bayes Nets

Part A - Bayesian Network:



CPT for M: $P(+m) = 0.3$, $P(-m) = 0.7$
CPT for L:

M	P(+l—M)
+m	0.9
-m	0.2

CPT for A:

M	L	P(+a—M,L)
+m	+l	0.8
+m	-l	0.6
-m	+l	0.1
-m	-l	0.01

Part B - Calculate P(+a):
Using marginalization, sum over all possible values of M and L:
$P(+a) = \sum_{m,l} P(+a|m,l) \cdot P(l|m) \cdot P(m)$

$$
\begin{aligned}
P(+a) &= P(+a|+m,+l) \cdot P(+l|+m) \cdot P(+m) &(1)\\
&+ P(+a|+m,-l) \cdot P(-l|+m) \cdot P(+m) &(2)\\
&+ P(+a|-m,+l) \cdot P(+l|-m) \cdot P(-m) &(3)\\
&+ P(+a|-m,-l) \cdot P(-l|-m) \cdot P(-m) &(4)
\end{aligned}
$$

Calculating each term:

- Term 1: $0.8 \times 0.9 \times 0.3 = 0.216$
- Term 2: $0.6 \times 0.1 \times 0.3 = 0.018$
- Term 3: $0.1 \times 0.2 \times 0.7 = 0.014$
- Term 4: $0.01 \times 0.8 \times 0.7 = 0.0056$

$P(+a) = 0.216 + 0.018 + 0.014 + 0.0056 = 0.2536$
Part C - Calculate P(+m—+a):
Using Bayes' rule: $P(+m|+a) = \frac{P(+a|+m) \cdot P(+m)}{P(+a)}$

First, find $P(+a|+m)$:

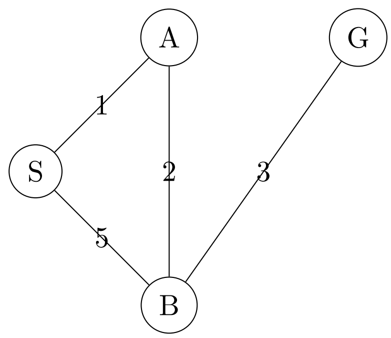
$$P(+a|+m) = P(+a|+m,+l) \cdot P(+l|+m) + P(+a|+m,-l) \cdot P(-l|+m) \tag{5}$$
$$= 0.8 \times 0.9 + 0.6 \times 0.1 \tag{6}$$
$$= 0.72 + 0.06 = 0.78 \tag{7}$$

Now apply Bayes' rule: $P(+m|+a) = \frac{0.78\times0.3}{0.2536} = \frac{0.234}{0.2536} = 0.923$

Problem 4: Graph Search and Optimality (BONUS)

Part A - Example Graph:

Consider this graph:



Heuristic values: h(S) = 4, h(A) = 4, h(B) = 1, h(G) = 0

Admissibility check:

- h(S) = 4, true cost = 6 (S → A → B → G) ✓
- h(A) = 4, true cost = 5 (A → B → G) ✓
- h(B) = 1, true cost = 3 (B → G) ✓
- h(G) = 0, true cost = 0 ✓

The heuristic is admissible.

Consistency check: Edge S → A: h(S) = 4, c(S,A) = 1, h(A) = 4 Need: $4 \leq 1 + 4 = 5$ ✓

Edge A → B: h(A) = 4, c(A,B) = 2, h(B) = 1 Need: $4 \leq 2 + 1 = 3$ **FALSE!**

The heuristic is **inconsistent**.

Part B - Graph Search Failure:

A* graph search trace:

1. Fringe: {S:4}, Closed: {}
2. Expand S: Fringe: {A:5, B:6}, Closed: {S}
3. Expand A (f=5): Try to add B with g=3, f=4, but B:6 is already in fringe

   In graph search, we keep B:6 in fringe (don't update to better path) Fringe: {B:6}, Closed: {S, A}
4. Expand B (f=6): Add G with g=5+3=8, f=8 Fringe: {G:8}, Closed: {S, A, B}
5. Expand G: Found goal with cost 8

Failure: Graph search found path S $\to$ B $\to$ G with cost 8, but optimal is S $\to$ A $\to$ B $\to$ G with cost 6.

The failure occurs because when we try to add B from A with a better g-value (3 vs 5), graph search doesn't update it since B is already in the fringe. With consistent heuristics, the first time we reach a node is guaranteed to be optimal, but with inconsistent heuristics, this guarantee breaks.